Testdokumentation

Nils Generlich, Yannik-Joel Herrchen, Nico Kroll, Paul Marx, Richard Schultz, Marlene Winter

28. Juni 2024

Inhaltsverzeichnis

1. Testkonzept: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04) . . . 1
1.1. Testdokumentation und -aufbau . . . 1
1.2. Teststrategie . . . 1
1.3. Testumgebung . . . 1
1.4. Testobjekte . . . 2
1.5. Testmethoden und -arten . . . 2
1.6. Teststufen . . . 2
1.7. Testorganisation und -durchführungsplanung . . . 3
2. Test Cases: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04) . . . 4
2.1. Test Case UC01 - Lose starten . . . 4
2.2. Test Case UC02 - Lose bewegen . . . 8
2.3. Test Case UC03 - Lose bearbeiten . . . 12
2.4. Test Case UC04/UC05 - Stammdaten anzeigen/bearbeiten . . . 18
2.5. Test Case UC06/UC07 - Daten exportieren/importieren . . . 19
2.6. Test Case UC08 - Backup erstellen . . . 21
2.7. Test Case: Addional Testing . . . 22
2.8. Test Case System-Wide Requirements . . . 23
3. Bugtracker - Fehlerliste: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04) . . . 28
4. Vereinfachte Testdatensätze: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04) . . . 30

1. Testkonzept: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04)

1.1. Testdokumentation und -aufbau

Die Testergebnisse wurden in separate Test-Cases gruppiert und dokumentiert. Für jeden **Test-Case** wurden Test-IDs definiert, mit deren Hilfe sich die einzelnen Tests eindeutig zuordnen lassen. Für jede Test-ID sind im Test-Case der Zweck, die Testart, die Testdurchführung, die Nachbedingungen, die Eingabedaten, das Ergebnis und eventuell nowendige To-dos sowie deren Korrekturstatus dokumentiert. Die Testergebnisse enthalten unter Umständen Terminalausgaben und Fehlermeldungen, die als Screenshots angehängt sind. Außerdem wurde für JUnit-Test die zugehörige Testabdeckung dokumentiert. Alle notwendigen Anpassungen wurden, um Übersichtlichkeit zu gewährleisten, parallel im **Bugtracker** dokumentiert. Hier werden auch die entsprechenden Test-IDs referenziert. Die verwendeten Testdatensätze der Datenbank wurden in der Datei **Test-Datensätze** festgehalten.

1.2. Teststrategie

Risikien und wichtige Systembestandteile

- Datenbankverbindung: Eine fehlerhafte Konfiguration der Datenbankverbindung kann das gesamte System unbrauchbar machen. Die korrekte Einstellung der Datenbankverbindung über die Config-Datei ist daher von höchster Priorität
- Benutzerverwaltung: Fehler in der Benutzerverwaltung, wie falsche Berechtigungen, können zu Sicherheitslücken führen und den Zugriff auf an Nutzerollen geknüpfte Uses-Cases ermöglichen
- Config-Datei: Die korrekte Funktion der Config-Datei ist essenziell, um systemweite Informationen wie Nutzerlogins und Datenbankverbindungen zu verwalten

Fehler effizient aufdecken

- Fokussierte Tests auf die kritischen Systemkomponenten wie zum Beispiel auf die Datenbankverbindung und die Benutzerrechte
- Einsatz von Unit-Tests für spezifische Funktionen wie generateLotID und alle timergesteuerten Funktionalitäten
- Integrationstests, um sicherzustellen, dass die verschiedenen Teile der Anwendung (Config-Datei, Datenbank, Benutzerverwaltung) nahtlos zusammenarbeiten

1.3. Testumgebung

Erforderliche Testumgebung und -infrastruktur:

- Verwendung von PostgreSQL als Datenbankmanagementsystem
- Enwicklungsumgebung wie Visual Studio Code oder PYCharme zur Entwicklung und Durchführung von Tests

Werkzeuge:

- unittest für Unit-Tests in Python → Import der Bibliothek "unittest" im jeweiligen Test-Case
- Zur Ausführung der Coverage-Test muss coverage installiert werden, z.B. mit "pip install coverage"
- Datenbankverwaltungs-Tools wie pgAdmin
- Asciidoc/Github zur Dokumentation

1.4. Testobjekte

Jeder zuvor definierte Use-Case wurde in einem eigenen Test-Case aufgegriffen. Die systemweite Anforderungen wurden in einem eigenen Dokument (test_case_system_wide_requirements.adoc) getestet. Ebenso wurden einzelne Funktionialitäten, die keinem UseCase zugeordnet werden konnten, getestet (test_case_additional_testing.adoc).

1.5. Testmethoden und -arten

Manuelle und automatisierte Tests

Alle Funktionalitäten, die zeitgesteuert vom Timer aufgerufen worden sind und nicht vom Nutzer gesteuert werden, wurden mithilfe von JUnit-Tests getestet. Dies sind inbesondere die Use-Cases 1-3 zum Starten, Bewegen und Bearbeiten der Lose im Fertigungsprozess. Manuelle Benutzerinteraktionen, wie das Erstellen eines Datenexports, wurden über den Aufruf der Webseite im Browser getestet. Außerdem wurde alle Änderungen in der Datenbank mit manuellen SQL-Abfragen mithilfe von pgAdmin plausibilisiert.

Positivtest und Negativtest

Zunächst wurden Positv-Tests durchgeführt, um grundsätzlich korrektes Systemverhalten sicherzustellen. Im Anschluss daran wurde die verbleibende Zeit darauf verwendet die Software auf Robustheit zu testen und absichtlich Fehler mithilfe von Negativ-Tests zu provozieren. Grundsätzlich wurden Positivtest und Negativtest für alle Test-Cases kombiniert verwendet.

Statischer Test und dynamischer Test

Zu Beginn von SE II wurde durch den Tester ein manuelles Review des Codes durchgeführt. Dabei wurden alle Python-Dateien auf Syntaxehler, fehlende Funktionalitäten und den Code-Stil untersucht. Potenzielle Probleme wurden entsprechend kommentiert und dem Entwickler-Team kommunziert. Im fortlaufenden Semester fanden vorwiegend dynamische Tests statt, wo bereits laufähige Software-Bestandsteile im Betrieb untersucht worden sind.

1.6. Teststufen

Modultest und Integrationstest

Wenn möglich, wurden einzelne Softwarkomponenten in Isolation getestet. Dadurch war es einfacher Fehler zu identifizieren, da der Testbereich nur auf ein einzelnes Modul beschränkt war.

Dies funktionierte insbesondere deswegen gut, weil die Fertigung in der Halbleiterfabrik sowieso modular abläuft und die Software zum Teil ein Abbild dieses modularen Fertigungsprozesses ist. Gegen Ende des Semesters wurde aber auch das Zusammenspiel aller einzelnen Softwarekomponenten mithilfe von Integrationstests geprüft. Dies war unbedingt notwendig, um sicherzustellen, dass sich die Lose korrekt vom Auftragseingang bis zum Ende des Fertgigungsprozesses bewegen. Am Ende des Semesters erfolgte ein Systemtest und Abnahmetest.

1.7. Testorganisation und -durchführungsplanung

Die Durchführung der Software-Tests wurden durch eine Einzelperson sichergestellt. Aufgrund des Workloads kam der effizienten Selbstorganisation und der Priorisierung der einzelnen Testcase eine besondere Rolle zu. Das bedeutet, dass die Test Cases entsprechend Ihrer in den Anforderungen spezifierten Priorität erstellt und abgearbeitet worden sind. Für die einzelnen Testcases wurde im Vorfeld jeder Testcase-Durchführung eine interene Liste mit allen zu testenden Funktionalitäten erstellt, die priorisiert und dieser Reihenfolge getestet worden ist. Hierbei sollte sichergestellt werden, dass die wichtigsten Fuktionalitäten zuerst getestet worden sind und zeitnah Feedback an die Entwickler gegeben werden konnte. Alle notwendigen Anpassung wurden für alle Teammitglieder sichtbar in der Datei "Bugtracker" kommunziert. Die Zeitplanung orientierte sich stets an den Kundenterminen, zu denen die Anwesenheit von Software-Fehlern des Prototypen ausgeschlossen werden musste. Grundsätzlich wurden die Tests unterteilt nach: UseCases (bspw. test_case_UC01_Los_starten.adoc), systemweiten Anforderungen (test_case_system_wide_requirements.adoc) und sonstigen Anforderungen (test_case_additional_testing.adoc).

2. Test Cases: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04)

2.1. Test Case UC01 - Lose starten

Tests mit Fokus auf folgende Funktionalitäten:

- Auftragsanlage
- Los starten (manuell + Timer)
- Datenvisualisierung über Los-Tabelle

Test-ID	1
Zweck	Verifikation Auftragsanlage
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt, sodass die Eingabemaske erscheint
Testdurchführung	1. Eingabe: Customer, Order, Product, Amount of wafer, Due date, Time interval 2. Auftragsanlage über Button "Register Order" abschließen 3. Prüfung, ob das Los im Zustand INIT in die Datenbank übertragen worden ist und alle Eingaben korrekt übertragen worden (SELECT * FROM LOT)
Nachbedingungen	- Bei erfolgreicher Testausführung befinden sich die zugehörigen Lose im Zustand INIT in der Datenbank - alle Wertefelder müssen korrekt in der Datenbank in der Tabelle "Lot" übertragen worden sein
Eingabedaten	Customer: Paul Order: Bestellung_2 Product: TestProductA Due date: 09,06.2024 Time interval: 08.05.2024-09,05.2024
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	2
Zweck	Verifikation Auftragsanlage
Testart	Manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt, sodass die Eingabemaske erscheint
Testdurchführung	1. Eingabe: Customer, Order, Product, Amount of wafer, Due date, Time interval 2. Auftragsanlage über Button "Register Order" abschließen 3. Prüfung, ob das System fehlende Eingaben erkennt 4. Prüfung, ob invalide Datumskombinationen erkannt werden (due date < start date, end date < start date, start date/current date/end date < current date)
Nachbedingungen	- Bei erfolgreicher Testausführung erscheinen deskriptive Fehlermeldungen. - das Los darf nicht auf der Datenbank eingeplant werden
Eingabedaten	Analog zu TID1
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	3
Zweck	Verifikation Los manuell starten
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt

Testdurchführung	1. Eingabe bzw. Auswahl eines Loses, welches gestartet werden soll und sich im Zustand INIT befindet 2. Los über den Button "Start Lot" starten 3. Prüfung, ob die zugehörige Los-Tabelle aktualisiert wird 4. Prüfung, ob Los-Zustand von INIT auf WAIT wechselt; Prüfung, ob Operation und Route gesetzt werden
Nachbedingungen	Bei erfolgreicher Testausführung wird das Los korrekt in den Fertigungsprozess eingebunden
Eingabedaten	Auswahl eines beliebigen Loses mit dem Zustand INIT
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	4
Zweck	Verifikation Los manuell starten
Testart	Manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt
Testdurchführung	1. Fehlerhafte Eingaben: Los nicht im Zustand INIT; Falsche LosID 2. Los über den Button "Start Lot" starten 3. Prüfung, ob deskriptive Fehlermeldungen ausgegeben werden
Nachbedingungen	Bei erfolgreicher Testausführung wurde kein fehlerhaftes Los gestartet
Eingabedaten	Auswahl eines beliebigen Loses mit dem Zustand !=INIT und Eingabe einer fehlerhaften Los ID (z.B. xy34950)
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	5

Zweck	Verifikation Filteroptionen der Los-Tabelle
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt
Testdurchführung	1. Diverse Filterkombinationen im Drop-Down-Menü auswählen 2. Kontrolle, ob die Datensätze korrekt gefiltert werden und sich die Los-Tabelle automatisch aktualisiert
Nachbedingungen	Bei erfolgreicher Testausführung befindet sich alle Lose, die den Filterkriterien entsprechen, in der Los-Tabelle
Eingabedaten	Auswahl aller Zustände in Kombinationen mit stichprobenartig ausgewählten Maschinen und Losen
Ergebnis	FAIL
To-dos	Das Filtern der Lose funktioniert nach dem Prinzip, dass alle Lose angezeigt werden, bei denen mindestens 1 Suchkriterium wahr ist. Es muss sichergestellt sein, dass alle Suchkriterien gleichzeitig zutreffen (siehe Screenshot).
Korrekturstatus	DONE

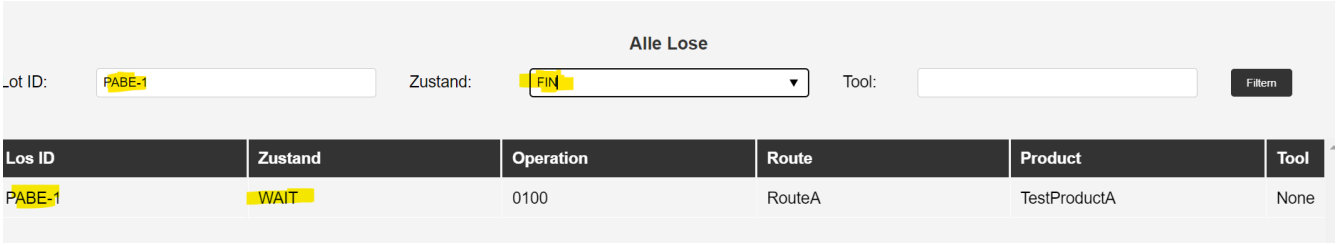


Abbildung 1. TID 5: Fehlerhafter Filter

Test-ID	6
Zweck	Verifikation Timer-Funktion Los starten
Testart	Unit-Test, Positiv-Test
Vorbedingungen	Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation

Testdurchführung	1. Ausführung des Moduls TC_UC1_TID6.py, welches die Timer-Funktion lot_start aufruft. 2. Lose im Zustand INIT einpflegen und Variable für das Startdatum in den gültigen Bereich setzen 3. Kontrolle, ob der Test erfolgreich durchläuft und die richtige Lose im Zustand INIT und WAIT ausgegeben werden
Nachbedingungen	Bei erfolgreicher Testausführung wechseln für das gewählte Startdatum alle Lose vom Zustand INIT in den Zustand WAIT
Eingabedaten	Anlegen eines Loses und Setzen eines Startdatums<currenttime
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

2.2. Test Case UC02 - Lose bewegen

Tests mit Fokus auf folgende Funktionalitäten:

- Los bewegen (manuell + Timer) und sauberes Setzen der Zustände WAIT und FIN
- Test auf das Bewegen auf optionale Operations

Test-ID	1
Zweck	Verifikation Los bewegen (Timer)
Testart	Unit-Test, Positiv-Test
Vorbedingungen	Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation

Testdurchführung	1. Ausführung des Moduls TC_UC2_TID1.py 2. Der Textcase generiert ein Test-Los zu Beginn des Fertigungsprozesses im Zustand OP_COMP 3. Nun wird das Test-Los automatisiert durch den Fertigungsprozess bewegt. Hierfür wird das Bewegen des Loses modular getestet und das Bearbeiten des Loses wird übersprungen. D.h. der Zustand des Loses wird von WAIT direkt auf OP_COMP gesetzt. 4. Das Los wird Schritt für Schritt bis ans Ende des Fertigungsprozesses bewegt. Dabei erfolgt eine Ausgabe, die den Weg des Loses durch den Fertigungsprozess visualisiert.
Nachbedingungen	Bei erfolgreicher Testausführung befindet sich das Los am Ende im Zustand FIN und wurde fehlerfrei durch die Fertigung bewegt.
Eingabedaten	TestProductA aus den Testdatensätzen im Zustand OP_COMP auf RouteA und Operation 0100.
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

```
Produktionsfluss von LotTestLotUC2ist zu Ende
Ausgabe TC | Lot ID: TestLotUC2, Operation: 0200, Route: RouteB, State: FIN
Ausgabe TC | Das Los hat erfolgreich den Fertigungsprozess durchlaufen.
.
----------------------------------------------------------------------
Ran 1 test in 0.186s

OK

[Done] exited with code=0 in 1.216 seconds
```

Abbildung 2. UC2 TID1: Pass

Coverage report: 86%

Files | Functions | Classes

coverage.py v7.5.1, created at 2024-05-11 10:41 +0200

File ▲	statements	missing	excluded	coverage
C:\Users\marxp\Documents\i04-projekt-frau-holle\src\database_connection.py	8	0	0	100%
C:\Users\marxp\Documents\i04-projekt-frau-holle\src\Lot_bew_bearb.py	165	36	0	78%
C:\Users\marxp\Documents\i04-projekt-frau-holle\src\mapper.py	56	0	0	100%
TC_UC2_TID1.py	32	0	0	100%
Total	**261**	**36**	**0**	**86%**

coverage.py v7.5.1, created at 2024-05-11 10:41 +0200

Abbildung 3. UC2 TID1: Coverage Report

Test-ID	2
Zweck	Verifikation Los bewegen; Ausführung von optionalen Operations
Testart	Unit-Test, Postiv-Test
Vorbedingungen	Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation
Testdurchführung	1. Der Textcase erhält die Los-ID eines zuvor in die Datenbank eingepflegten Test-Loses 2. Eingabe der optionalen Operation und der Ausführungswahrscheinlichkeit in die jeweiligen Variablen 3. Ausführung des Moduls TC_UC2_TID2.py 4. Nun durchläuft das Test-Los 100 Mal das Bewegen vor einer optionalen Operation. Die jeweiligen Operations werden ausgegeben.
Nachbedingungen	Bei erfolgreicher Testausführung wird das Los mit einer bestimmten Wahrscheinlichkeit auf die optionale Operation bewegt. Der Test gilt als bestanden, wenn die theoretische Ausführungswahrscheinlichkeit und die tatsächliche Ausführungswahrscheinlichkeit ein Delta kleiner 10% aufweist.
Eingabedaten	TestProductA aus den Testdatensätzen im Zustand OP_COMP auf RouteA und Operation 0200.
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

```
Tats�chliche Anzahl Ausf�hrungen: 23
Erwartete Anzahl an Ausf�hrungen: 20
.
----------------------------------------------------------------------
Ran 1 test in 2.119s

OK

[Done] exited with code=0 in 2.967 seconds
```

Abbildung 4. UC2 TID2: Pass

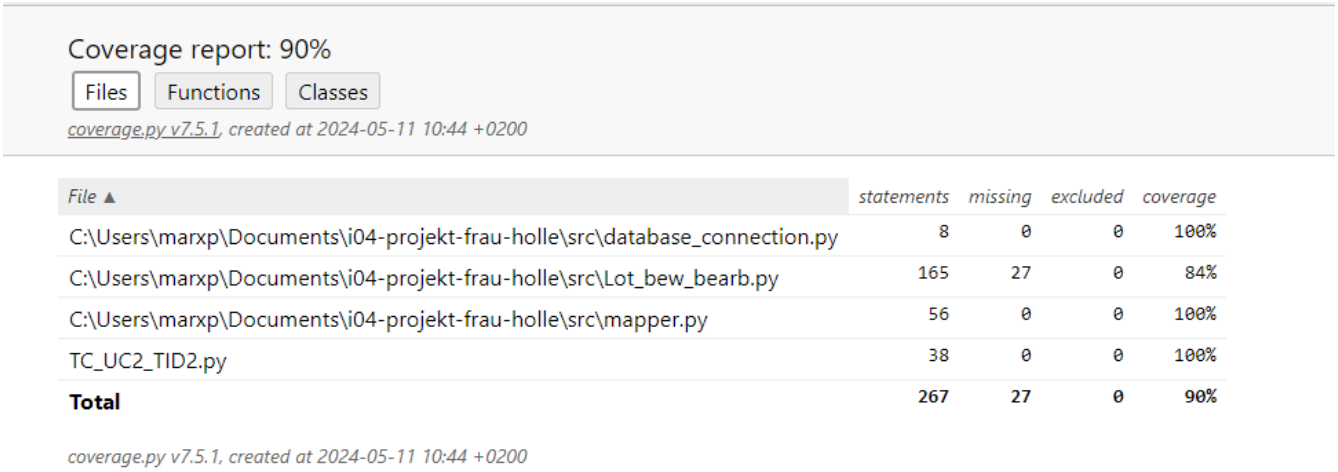


Abbildung 5. UC2 TID2: Coverage Report

Test-ID	3
Zweck	Verifikation Los bewegen (manuell)
Testart	manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los bewegen" wurde angeklickt, sodass die Eingabemaske erscheint - das Test-Los befidnet sich im Zustand OP_COMP
Testdurchführung	1. Erstellung eines Test-Loses 2. Bewegen des Loses durch den Fertigungsprozess durch mehrfaches Drücken des Buttons "Los bewegen" 3. Prüfung des Los-Zustandes, des Zeitstempels (enteroper), der Route und der Operation auf der Datenbank mithilfe von: SELECT * FROM LOT WHERE lotid='PATE-14'. Der Abgleich erfolgt anhand der Testdatensätze und den dort hinterlegten Fertigungsplänen. 4. Setzen des Los-Zustandes auf OP_COMP mithilfe von UPDATE lot SET "STATE"='OP_COMP' WHERE lotid='PATE-14'. Hierdurch wird immer wieder die Vorbedingung für das Bewegen des Loses geschaffen. 5. Springe zu Punkt 2 bis das Los das Ende des Fertigungsprozesses erreicht
Nachbedingungen	Bei erfolgreicher Testausführung befindet sich das Los am Ende im Zustand FIN und wurde fehlerfrei durch die Fertigung bewegt.

Eingabedaten	lotid=PATE-14 und product=TestProductA aus den Testdatensätzen im Zustand OP_COMP auf RouteA und Operation 0100.
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	4
Zweck	Verifikation Los bewegen (manuell)
Testart	manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los bewegen" wurde angeklickt, sodass die Eingabemaske erscheint - Lose mit einem Zustand != OP_COMP
Testdurchführung	1. Erstellung eines Test-Loses 2. Los bewegen ausführen, obwohl sich das Los nicht im Zustand OP_COMP befindet und somit nicht bewegt werden darf
Nachbedingungen	Bei erfolgreicher Testausführung wird das Los nicht bewegt und es erscheint eine deskriptive Fehlermeldung.
Eingabedaten	Verschiede Los-IDs mit unterschiedlichen Los-Zuständen.
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

2.3. Test Case UC03 - Lose bearbeiten

Tests mit Fokus auf folgende Funktionalitäten:

- Los bearbeiten (manuell + Timer) und sauberes Setzen der Maschinen- und Loszustände
- Tests auf spezifische Fehlerfälle (z.B. keine Maschine verfügbar)

Test-ID	1
Zweck	Verifikation Los bearbeiten (Timer)
Testart	Unit-Test, Positiv-Test

Vorbedingungen	- Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation - kein anderes Los darf sich vor TMachineA im Zustand WAIT befinden
Testdurchführung	1. Ausführung des Moduls TC_UC3_TID1.py 2. Der Testcase legt ein Test-Los in der Datenbank im Zustand WAIT an 3. Der Zustand IDLE wird für das jeweilige Test-Tool gesetzt 4. Die Funktion tool_searches_for_lots() wird ausgeführt und im Anschluss schläft der Prozess bis die Bearbeitung abgeschlossen ist 5. Abgleich des Los- und Maschinenzustands
Nachbedingungen	Der Test wurde erfolgreich ausgeführt, wenn das Test-Los nach der Bearbeitung in den Zustand OP_COMP und die Test-Maschine in den Zustand WAIT gewechselt ist
Eingabedaten	Ein Test-Los mit TestProductA aus den Testdatensätzen im Zustand WAIT auf RouteA und Operation 0100.
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

```
Checking if a tool is dones\104-projekt-frau-holle\104-projekt-frau-holle\src\testing>
Searching for tools that are IDLE and waiting for lots
Checking if a lot is in OP_COMP state
Zustand des Test-Loses VOR Ausführung von lotop_finished: WAIT
Zustand von TMachineA VOR Ausführung von lotop_finished: IDLE
Searching for tools that are IDLE and waiting for lots
There are lots waiting for the tool TMachineA in IDLE state
Lot TestLotUC44 is selected to be processed by tool TMachineA
Lot TestLotUC44 is PROC now
Tool TMachineA is PROC now

Checking if a tool is done
Tool TMachineA is done (PROC -> IDLE)
Lot TestLotUC44 is OP_COMP now

Zustand des Test-Loses nach Ausführung von lotop_finished: OP_COMP
Zustand der Maschine nach Ausführung von lotop_finished: IDLE
Der Test ist erfolgreich.
.
----------------------------------------------------------------------
Ran 1 test in 20.169s

OK
```

Abbildung 6. UC3 TID1: PASS und Testausgaben

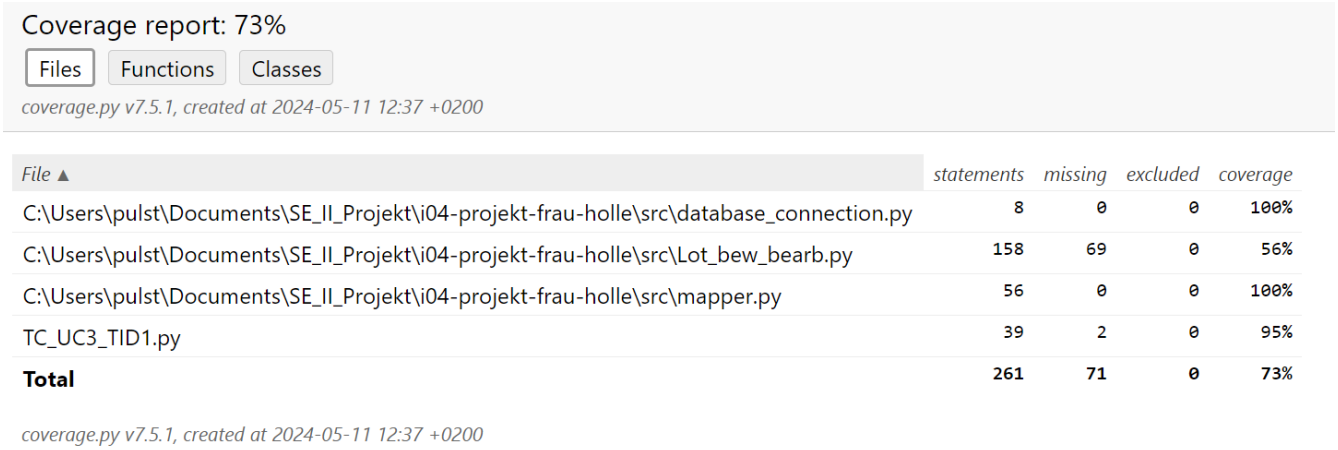


Abbildung 7. UC3 TID1: Coverage Report

Test-ID	2
Zweck	Korrekte Los-Auswahl beim Bearbeiten eines Loses (FiFo-Prinzip)
Testart	Unit-Test, Positiv-Test
Vorbedingungen	- Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation - kein anderes Los darf sich vor TMachineA im Zustand WAIT befinden
Testdurchführung	1. Ausführung des Moduls TC_UC3_TID2.py 2. Der Testcase legt ein Test-Los-1 in der Datenbank im Zustand WAIT an und legt sich schlafen → ein erster Zeitstempel für ENTEROPER ensteht 3. Der Testcase legt ein Test-Los-2 in der Datenbank im Zustand WAIT an und legt sich schlafen → ein zweiter Zeitstempel für ENTEROPER entsteht + 4. Der Zustand IDLE für das jeweilige Test-Tool gesetzt 5. Im Anschluss wird der Zustand des Test-Loses 1 geprüft Abgleich des Los- und Maschinenzustands
Nachbedingungen	Der Test gilt als erfolgreich, wenn das Test-Los-1 für die Bearbeitung ausgewählt worden ist (FiFo-Prinzip) und sich folglich im Zustand PROC befindet.
Eingabedaten	Zwei Test-Lose mit TestProductA aus den Testdatensätzen im Zustand WAIT auf RouteA und Operation 0100.
Ergebnis	PASS

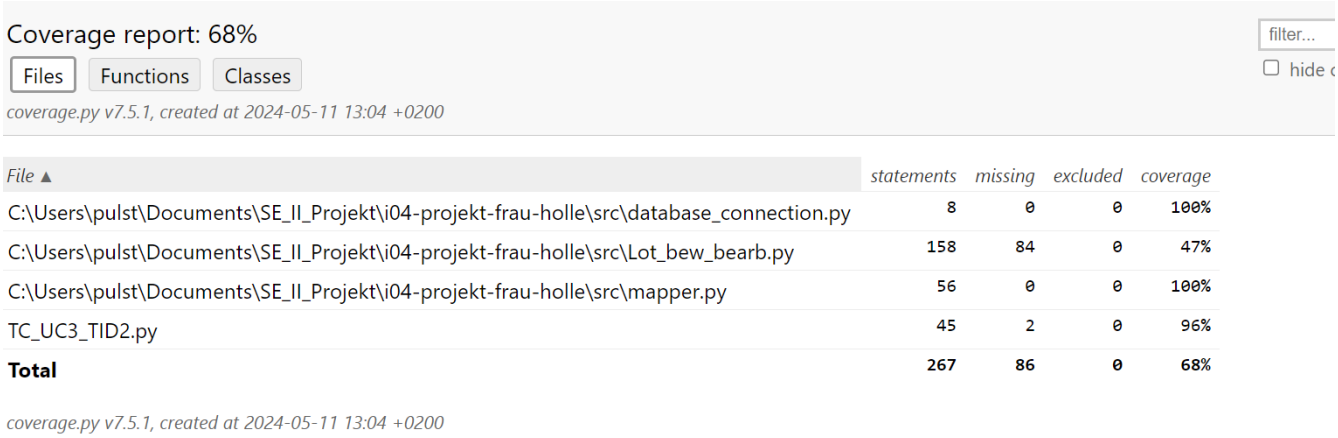
To-dos	/
Korrekturstatus	/

```
Checking if a tool is done
Tool TMachineA is not done

Searching for tools that are IDLE and waiting for lots
Checking if a lot is in OP_COMP state
Zustand des ersten Test-Loses: WAIT
Zustand des zweiten Test-Loses: WAIT
Searching for tools that are IDLE and waiting for lots
There are lots waiting for the tool TMachineA in IDLE state
Lot TestLotUC44 is selected to be processed by tool TMachineA
Lot TestLotUC44 is PROC now
Tool TMachineA is PROC now

Zustand des ersten Test-Loses nach tool_searches_for_lots: PROC
Zustand des zweiten Test-Loses nach tool_searches_for_lots: WAIT
Zustand von TMachineA nach tool_searches_for_lots: PROC
Der Test ist erfolgreich.
.
----------------------------------------------------------------------
Ran 1 test in 5.145s
```

Abbildung 8. UC3 TID2: PASS und Testausgaben



Coverage report: 68%

Files | Functions | Classes

filter... hide

coverage.py v7.5.1, created at 2024-05-11 13:04 +0200

File ▲	statements	missing	excluded	coverage
C:\Users\pulst\Documents\SE_II_Projekt\i04-projekt-frau-holle\src\database_connection.py	8	0	0	100%
C:\Users\pulst\Documents\SE_II_Projekt\i04-projekt-frau-holle\src\Lot_bew_bearb.py	158	84	0	47%
C:\Users\pulst\Documents\SE_II_Projekt\i04-projekt-frau-holle\src\mapper.py	56	0	0	100%
TC_UC3_TID2.py	45	2	0	96%
Total	**267**	**86**	**0**	**68%**

coverage.py v7.5.1, created at 2024-05-11 13:04 +0200

Abbildung 9. UC3 TID2: Coverage Report

Test-ID	3
Zweck	Los bearbeiten (manuell) prüfen
Testart	manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Los bearbeiten" wurde angeklickt, sodass die Eingabemaske erscheint - Lose befindet sich im Zustand WAIT und zugehörige Maschinen im Zustand IDLE

Testdurchführung	1. Auswahl eines Loses im Zustand WAIT im Drop-Down-Menü. 2. Drücken des Buttons "bearbeiten" 3. Abgleich des Los- und Maschinenzustands
Nachbedingungen	Der Test gilt als erfolgreich, wenn die SUCCESS-Page erscheint und sich das Los und die Maschine im Zustand PROC befindet
Eingabedaten	Beliebiges Los, welches den Vorbedingungen entspricht
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	4
Zweck	Los bearbeiten (manuell) prüfen; keine Maschine verfügbar
Testart	manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Los bearbeiten" wurde angeklickt, sodass die Eingabemaske erscheint - Lose befindet sich im Zustand WAIT und zugehörige Maschine nicht im Zustand IDLE
Testdurchführung	1. Auswahl eines Loses im Zustand WAIT im Drop-Down-Menü. 2. Drücken des Buttons "bearbeiten" 3. Abgleich des Loszustands
Nachbedingungen	Der Test gilt als erfolgreich, wenn die FAILURE-Page erscheint und das Los nicht bearbeitet werden konnte
Eingabedaten	Beliebiges Los, welches den Vorbedingungen entspricht
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

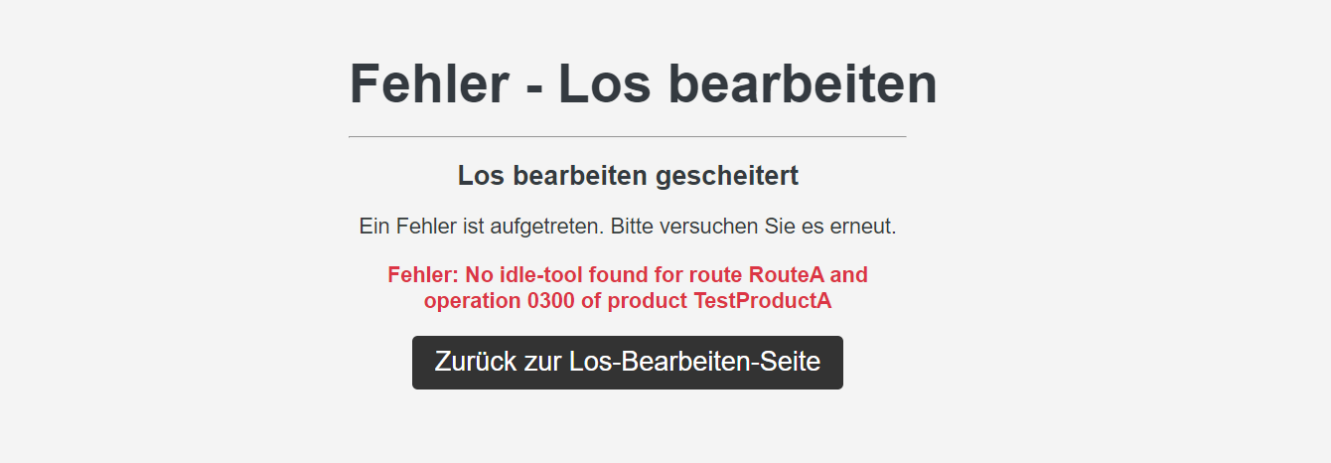


Abbildung 10. UC3 TID4: korrekte Failure-Page

Test-ID	5
Zweck	Los bearbeiten (manuell) prüfen, Los nicht im Zustand WAIT
Testart	manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Los bearbeiten" wurde angeklickt, sodass die Eingabemaske erscheint + - Lose befindet sich nicht im Zustand WAIT
Testdurchführung	1. Auswahl eines Loses im Zustand !WAIT im Drop-Down-Menü. 2. Drücken des Buttons "bearbeiten" 3. Abgleich des Loszustands
Nachbedingungen	Der Test gilt als erfolgreich, wenn die FAILURE-Page erscheint und das Los nicht bearbeitet werden konnte
Eingabedaten	Beliebiges Los, welches den Vorbedingungen entspricht
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

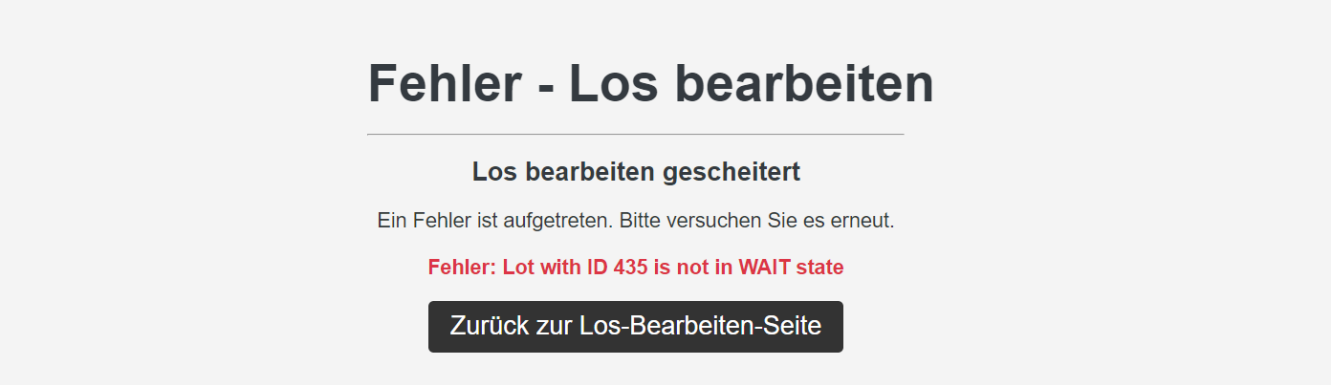


Abbildung 11. UC3 TID5: korrekte Failure-Page

2.4. Test Case UC04/UC05 - Stammdaten anzeigen/bearbeiten

Tests mit Fokus auf folgende Funktionalitäten:

- Anzeige von Stammdaten
- Bearbeitung von Stammdaten

Test-ID	1
Zweck	Verifikation Stammdaten bearbeiten/anzeigen
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Stammdaten" wurde angeklickt, sodass die Eingabemaske erscheint
Testdurchführung	1. Auswahl einer Lot ID und Bestätigung über EditLot 2. Prüfung, ob die Wertfelder korrekt befüllt werden 3. Veränderung aller Wert-Felder (ID, Priority, Pieces, State, Product, Route, Operation, Customer, Order, Tool display, Duedate, Startdate, Operion entered, Process started) 4. Kontrolle der veränderten Datensätze in der Anzeige-Tabelle und über SELECT-Befehl in pgAdmin
Nachbedingungen	Bei erfolgreicher Testausführung konnten alle Attribute des Loses erfolgreich geändert und im Anschluss über die Tabelle ausgegeben werden. Außerdem wurden die Wertfelder zu Beginn der Bearbeitung korrekt befüllt und stimmen mit den Datensätzen aus der Los-Tabelle überein.

Eingabedaten	ID: Test394 Priority: 1 Pieces: 59 State: WAIT Product : TestProductA Order: Testorder4 Route: TestRoute409 Customer: Paul Due date: 09.06.2024 Start date: 09.06.2024 Operation enterd: 08.05.2024-09,05.2024 Process started: 08.05.2024-09,05.2024
Ergebnis	FAIL
To-dos	- Invalid Data Format beim Eintragen von Process started und EnterOper. Der Zeitstempel enthält zusätzlich Stunden und Minuten und muss entsprechend angepasst werden. - Product-Änderung lädt ohne Ergebnis
Korrekturstatus	DONE

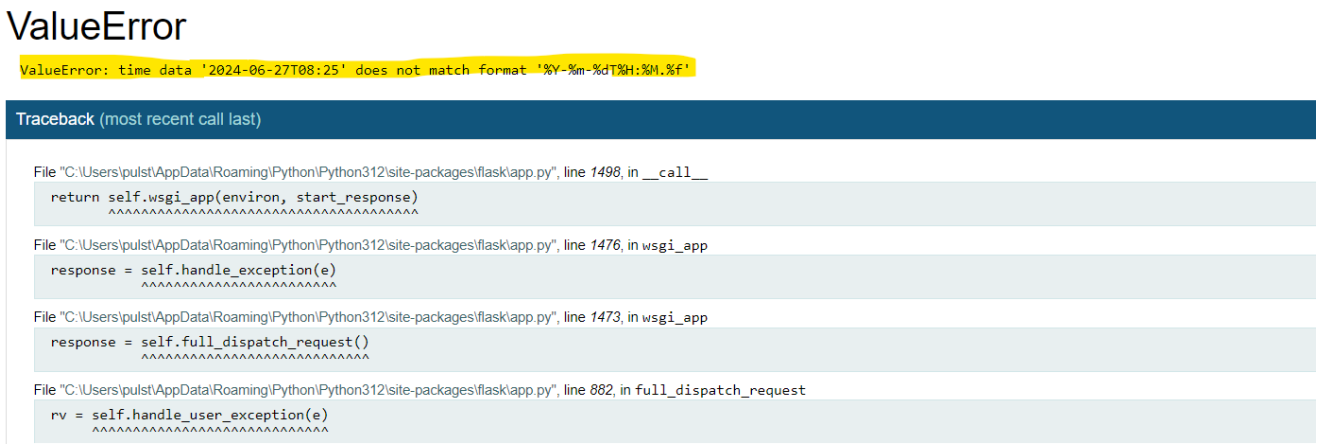


Abbildung 12. TID 1: Fehlermeldung TimeStamp

2.5. Test Case UC06/UC07 - Daten exportieren/importieren

Tests mit Fokus auf folgende Funktionalitäten:

- Datenexport
- Datenimport

Test-ID	1

Zweck	Verifikation Daten exportieren
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Daten exportieren" wurde angeklickt
Testdurchführung	1. Auswahl diverser LotID über Radiobutton 2. Datensätze exportieren 3. Kontrolle der erstellten Datei auf Korrektheit und Vollständigkeit der Daten sowie korrektes Dateiformat
Nachbedingungen	Bei erfolgreicher Testausführung wurde eine CSV-Datei mit Header erstellt und alle Daten, die zuvor ausgewählt worden sind, in diese Datei geschrieben
Eingabedaten	Auswahl von 3-5 Losen
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	2
Zweck	Verifikation Daten importieren
Testart	Manuell, Negativ-Test
Vorbedingungen	- Fertigungsleiter ist in der Web-Anwendung angemeldet - der Button "Daten importieren" wurde angeklickt - pgAdmin wird ausgeführt
Testdurchführung	1. Auswahl von verschiedenen CSV-Dateien 2. Klicken des Buttons "Hinzufügen" 3. SELECT-Befehl auf Los-Tabelle in pgAdmin zur Kontrolle, ob die Daten erfolgreich importiert worden sind
Nachbedingungen	Bei erfolgreicher Testausführung wurden die Daten der CSV-Datei erfolgreich in die Datenbank übertragen
Eingabedaten	- Exportierte Lostabelle aus unserem System - Lostabelle mit den Beispieldatensätzen von Herr Ringel
Ergebnis	FAIL

To-dos	- Bei der **exportieren Lostabelle aus unserem System** kein Import möglich: Header beim Export sollte nach Möglichkeit den Beispieldatensätzen von Ringel entsprechen → sonst kein Import. Das bedeutet: Exportdateien brauchen Spaltenüberschriten in Großbuchstaben und keine Leerzeichen zwischen den Kommata im Header - Fehlermeldungen überschneiden sich teilweise. Zum Beispiel, wenn ein Import mit einem falschen Header stattfindet (z.B. Export aus unserem System), dann gibt es eine Fehlermeldung und der Vorgang wird abgebrochen. Gleichzeitig wird die Ausgabe "File successfully imported" angezeigt.
Korrekturstatus	IN PROGRESS



Abbildung 13. TID 2: Fehldermeldung beim Import von einem Export aus unserem System inkl. sich überschneidender Fehlermeldungen

2.6. Test Case UC08 - Backup erstellen

Tests mit Fokus auf folgende Funktionalitäten:

- Backup erstellen

Test-ID	1
Zweck	Verifikation Backup erstellen
Testart	Manuell, Positiv-Test

Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - pgAdmin wird ausgeführt
Testdurchführung	1. Starten des Backups durch Klicken des Buttons "Backup erstellen" 2. Kontrolle der Anzahl von Datensätzen in allen Tabellen über SQL-Befehl in pgAdmin 3. Prüfung, ob das Backup korrekt im vorgesehenen Verzeichnis hinterlegt worden ist 4. Prüfung der Daten innerhalb der Backup-Dateien auf Korrektheit und Vollständigkeit
Nachbedingungen	Bei erfolgreicher Testausführung befinden sich alle Backupdateien im Backup-Verzeichnis und die Anzahl der Datensätze entspricht den zuvor festgestellten Datensätzen.
Eingabedaten	Zählen der Beispieldatensätze: z.B.: SELECT COUNT(*) FROM lot;
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

2.7. Test Case: Addional Testing

Tests mit Fokus auf folgende Funktionalitäten:

- zusätzliche Softwarebestandteile, die keine systemweiten Anforderungen und keinem UseCase zugehörig sind

Test-ID	1
Zweck	Verifikation diverser Filter- und Sortieroptionen
Testart	Manuell, Positiv-Test
Vorbedingungen	Fertigungsleiter ist in der Web-Anwendung angemeldet
Testdurchführung	1. Auswahl divereser Filter- und Sortieroptionen 2. Prüfung, ob die zugehörigen Ausgabetabellen die gesetzte Auswahl korrekt umsetzt
Nachbedingungen	Bei erfolgreicher Testausführung wurde die zugehörige Ausgabetabelle nach bestimmten Merkmalen sortiert/gefiltert

Eingabedaten	z.B. absteigende LosID und Zustand WAIT im UseCase "Lost bearbeiten"
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

2.8. Test Case System-Wide Requirements

Tests mit Fokus auf folgende Funktionalitäten:

- funktionale Anforderungen
- nicht-funktionale Anforderungen

Test-ID	1
Zweck	Verifikation SWFA-1
Testart	Manuell, Negativ-Test
Vorbedingungen	- Login-Fenster der Anwendung ist aufgerufen
Testdurchführung	1. Eingabe von Nutzerdaten für den Nutzer/Mitarbeiter 2. Versuch, Use-Cases auszuführen, die nicht der Nutzerrolle entsprechen. Zum Beispiel das Importieren von Daten.
Nachbedingungen	- Bei erfolgreicher Testausführung konnte keine UseCases ausgeführt werden, der nicht der Nutzerrolle entspricht
Eingabedaten	user: user2 Passwort: pass2
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	2
Zweck	Verifikation SWFA-2
Testart	Manuell, Positiv-Test

Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt, sodass die Eingabemaske erscheint
Testdurchführung	1. Dokumentation der Kennzahlen des Cockpits 2. Nach 30 Minuten erneute Dokumentation der Kennzahlen des Cockpits und Abgleich mit dem vorherigen Stand
Nachbedingungen	Bei erfolgreicher Testausführung hat der Timer erfolgreich die Produktionsprozesse angestoßen und die Los- und Maschinenzustände haben sich geändert
Eingabedaten	/
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	3
Zweck	Verifikation SWFA-3
Testart	Manuell, Positiv-Test
Vorbedingungen	- Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation mit python app.py - Aufruf von pgAdmin zur Administration der Datenbank
Testdurchführung	1. Einfügen eines Test-Loses mittels SQL 2. Neustart des Systems 3. Anzeigen des Test-Loses mittels SELECT-Befehl in pgAdmin
Nachbedingungen	Bei erfolgreicher Testausführung wurde das Test-Los persistens in der Datenbank gespeichert
Eingabedaten	INSERT INTO LOT VALUES ('TestLot', 44, 10, 'WAIT', 'TestProductA', 'RouteA', '0100', 'TestCustomer', '2024-07-06', '2024-02-02', NULL, NULL, NULL, NULL);
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	4
Zweck	Verifikation SWFA-4
Testart	Manuell, Positiv-Test
Vorbedingungen	- Aufruf des Projektes in der Entwicklungsumgebung - Aufruf der Datei config.py
Testdurchführung	1. Änderung der Variable PASSWORD_USER1 für user1 2. Start der Anwendung durch python app.py 3. Login in der Webanwendung durch Verwendung des neuen Passworts, welches in der Config-Datei eingestellt worden ist
Nachbedingungen	Bei erfolgreicher Testausführung kann sich user1 erfolgreich anmelden
Eingabedaten	PASSWORD_USER1=root
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	5
Zweck	Verifikation QAGU-1
Testart	Manuell, Positiv-Test
Vorbedingungen	- Fertigungsleiter oder Mitarbeiter ist in der Web-Anwendung angemeldet - der Button "Los starten" wurde angeklickt, sodass die Eingabemaske erscheint
Testdurchführung	1. Prüfung aller Buttons, ob Doppelbelegung vorkommen 2. Prüfung, ob Spaltennamen der Tabellen eindeutig 3. Prüfung, ob Filter deskriptiv benannt sind
Nachbedingungen	Bei erfolgreicher Testausführung ist die Usability des Systems gegeben und die Gesamtheit der Interaktionselemente ist minimiert
Eingabedaten	/
Ergebnis	PASS
To-dos	/

Korrekturstatus	/

Test-ID	6
Zweck	Verifikation QAGZ-1
Testart	Manuell, Positiv-Test
Vorbedingungen	- Aufruf des Projektes in der Entwicklungsumgebung und Start der Applikation mit python app.py
Testdurchführung	1. Dokumentation ausgewählter Maschinen- und Loszustände 2. Erzwingen eines Verbindungsabbruches 3. 5 Minuten später erfolgt das Wiederherstellen der Verbindung 4. Kontrolle der dokumentierten Maschinen- und Loszustände
Nachbedingungen	Bei erfolgreicher Testausführung stimmen die Daten überein und das System konnte die Konsistenz der Daten gewährleisten
Eingabedaten	/
Ergebnis	PASS
To-dos	/
Korrekturstatus	/

Test-ID	7
Zweck	Verifikation QAGW-1:
Testart	Manuell, Positiv-Test
Vorbedingungen	- Aufruf github Repository
Testdurchführung	1. Prüfung der Entwickler - und Betriebsdokumentation 2. Kontrolle auf Vollständigkeit, Korrektheit und Verständlichkeit
Nachbedingungen	Bei erfolgreicher Testausführung ist durch die umfangreiche Dokumenation eine gute Wartbarkeit der Anwendung und ein gutes Verständnis bei auftretenden Fehlern gewährleistet
Eingabedaten	/
Ergebnis	PASS

To-dos	/
Korrekturstatus	/

3. Bugtracker - Fehlerliste: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04)

Der Bugtracker dient der schnellen Erfassung aller aufgetretenen Fehler und hält den jeweiligen Bearbeitungsstatus fest. Behobene Fehler mit [DONE] gekennzeichnet. Nicht behobene Fehler werden fettgedruckt. Anhand der dokumentierten TestCases kann bei Bedarf mithilfe der Test-ID der konkrete Test inkl. Eingabedaten u.v.m. eingesehen werden. Der Bugtracker dokumentiert zum Abschluss des Projektes ebenso den Ist-Fehlerzustand des Systems. Dies ist wichtig, weil die Anwendung zukünftig weiterentwickelt werden soll.

Test Case UC01 - Lose starten

- TID 5: Filterfunktionen [DONE]

Test Case UC02 - Lose bewegen

- Fehlende Funktionalität: Lose, die nach der Produktion auf HOLD gesetzt worden sind, müssen manuell geprüft werden. Der Use-Case muss es ermöglichen, dass der Folgezustand dieser Lose manuell auf SCRAP oder OP_COMP gesetzt werden kann (siehe im unteren Bereich von docs/test/docs/requirements/use_case_02_Hinweise_Implementierung.adoc.). Wir haben die UseCases etwas abweichend (aber gut!) von den Wireframes implementiert. Ursprünglich war ein LineEdit gedacht. Einfacher ist eventuell eine Lösung mit einem Dropdown mit allen Losen im Zustand HOLD. Wahlweise dann mit einem weiteren Dropdown der Zustände HOLD/SCRAP und einem Button zum Absenden oder kein Dropdown mit zwei unterschiedliche Button zum Setzen des Zustandes SCRAP oder OP_COMP. [DONE]

Test Case UC03 - Lose bearbeiten

- **Nach Möglichkeit die grafische Oberfläche anpassen: Tabellenbreite expandieren und Trennlinien wie bei "Los starten" einfügen**
- Datenbank mit inkosistenten Variablen: Lot-Tabelle mit Distool|varchar(10) und Tool-Tabelle mit Tool|varchar(15) → kann dazu führen, dass korrekt eingepflegte Maschinen der Tool-Tabelle das "Los bearbeiten" zum Absturz bringen [DONE]

Test Case UC04 - UC05 Stammdaten anzeigen/bearbeiten

- TID1: Invalid Data Format beim Eintragen von Process started und EnterOper [DONE]
- TID1: Customer lässt sich bei mir nicht editieren und lädt ohne Ergebnis [DONE]
- NiceToHave: Tabellenhöhe auf mehr als 3 Zeilen erweitern → bessere Lesbarkeit [DONE]
- NiceToHave: Auto-Refresh der Tabelle, wenn der Button Submit gedrückt wird → bessere Usability [DONE]

Test Case UC06-UC07 Daten importieren/eportieren

- **TID2: Header beim Export sollte nach Möglichkeit den Beispieldatensätzen von Ringel**

entsprechen, sonst ist ein Import von exportierten Dateien aus unserem System nicht möglich. Das bedeutet: Exportdateien brauchen Spaltenüberschriten in Großbuchstaben und keine Leerzeichen zwischen den Kommata im Header

- **TID2: beim Datei-Export (manuell) sind Spalten markiert, die laut Beschreibung ausgefüllt werden müssen. Es ist aber möglich z.B. ein Lot nur mit seiner ID einzupflegen. Hier müssten wir uns entscheiden, ob wir die Validierung der Eingaben ev. lieber an die Folgegruppe abgeben (ich denke, das wäre aus Zeigründen sinnvoll). Hier müsste eine Menge geprüft werden: Zustände dürften nur den definierten Zuständen entsprechen. Ein Los im Zustand INIT hat kein ENTEROPER oder PROCSTART. Das Feld müsste also leer sein. Ein Los im Zustand PROC muss aber ein PROCSTART besitzen etc.**
- **Fehlermeldungen überschneiden sich teilweise. Zum Beispiel, wenn ein Import mit einem falschen Header stattfindet (z.B. Export aus unserem System), dann gibt es eine Fehlermeldung und der Vorgang wird abgebrochen. Gleichzeitig wird die Ausgabe "File successfully imported" angezeigt.**



- **NiceToHave: Zusätzlich Ausgabe der Zeilennummer bei Fehlermeldungen → gerade bei großen Exports sind Fehler einfacher zu finden und zu korrigieren**
- **NiceToHave: Genauere Spezifikation, wie Header aufgebaut sein muss beim Datei-Import (z.B. Spaltenüberschriften in Großbuchstaben und durch Kommata getrennt; keine Leerzeichen erlaubt)**

4. Vereinfachte Testdatensätze: Modell eines automatisierten Fabrikbetriebes - Frau Holle (I04)

Die vereinfachten Testdatensätze werden benötigt, um die JUnit-Tests auszuführen. Außerdem dienen sie dazu den Fertigungsablauf transparent zu gestalten, da die verwendeten Routes und Operations auf ein Minimum reduziert sind. Weiterhin stellen die Testdatensätze definitiv einen reibungslosen Fertigungsprozess sicher und enthalten keine fehlerhaften Daten.

```
INSERT INTO PRODFLOW VALUES

('TestProductA', 'RouteA', 1),
('TestProductA', 'RouteB', 2);

INSERT INTO WORKFLOW VALUES

('RouteA', '0100', 'initial checking', 1, NULL),
('RouteA', '0200', 'processing', 1, NULL),
('RouteA', '0300', 'fnishing', 0, '20.00'),
('RouteB', '0100', 'pre proc', 1, NULL),
('RouteB', '0200', 'proc', 1, NULL);

INSERT INTO TOOL VALUES
('TMachineA', 'ALN1000', 'Robert', 'Part', 6, 100, 'IDLE'),
('TMachineB', 'ALN1000', 'Robert', 'Part', 6, 100, 'IDLE');

INSERT INTO PTIME VALUES
('TMachineA', 'TestProductA', 'RouteA', '0100', 15, 0),
('TMachineA', 'TestProductA', 'RouteA', '0200', 15, 0),
('TMachineA', 'TestProductA', 'RouteA', '0300', 15, 0),
('TMachineB', 'TestProductA', 'RouteA', '0100', 15, 0),
('TMachineB', 'TestProductA', 'RouteA', '0200', 15, 0),
('TMachineB', 'TestProductA', 'RouteA', '0300', 15, 0),
('TMachineA', 'TestProductA', 'RouteB', '0100', 15, 0),
('TMachineA', 'TestProductA', 'RouteB', '0200', 15, 0);

INSERT INTO ENTITIES VALUES
('TMachineA', 'TOOL', 1, 'a VWP tool', 'REG'),
('TMachineB', 'TOOL', 1, 'a VWP tool', 'REG');

INSERT INTO LOT VALUES
('TestLot', 44, 10, 'WAIT', 'TestProductA', 'RouteA', '0100', 'TestCustomer', '2024-
07-06', '2024-02-02', NULL, NULL, NULL, NULL);
```